🗓️ Planning d'implémentation — Assistant Matanne

> **Source** : `ANALYSE_COMPLETE.md` — généré le 28 mars 2026
> **Couverture** : Intégralité des sections 2 à 13 de l'analyse + annexes
> **Légende efforts** : XS ≤ 1h | S = 2-4h | M = 1-2j | L = 3-5j

Table des matières

1. Vue d'ensemble & Roadmap
2. Sprint Correctif — Bugs actifs
3. Sprint SQL — Consolidation
4. Sprint Tests — Couverture manquante
5. Sprint Documentation
6. Sprint 11 — Features manquantes prioritaires
7. Sprint 12 — Architecture & Refactoring
8. Sprint 13 — WhatsApp étendu + Email complet
9. Sprint 14 — Nouvelles opportunités IA + Cron jobs manquants
10. Sprint 15 — Interactions inter-modules avancées
11. Sprint 16 — Admin complet
12. Dépendances entre items
13. Suivi d'avancement global

1. Vue d'ensemble & Roadmap

État actuel

Dimension	État	Score
Couverture fonctionnelle	70 pages, 28 routes API, ~130 tables	✅ ~95%
Bugs bloquants	2 critiques, 3 hauts, 8 moyens	🔴 À corriger

Dimension	État	Score
Couverture tests	~786 tests API, ~200+ tests services	🟡 Manques ciblés
Documentation	13 docs techniques + guides	🟡 Partiellement stale
SQL consolidation	INIT_COMPLET.sql v3 (~130 tables)	🟡 V002 intégré
IA intégration	Mistral + Multimodal (Pixtral) actifs	✅ Bien intégré
Cron jobs	19 jobs APScheduler + 6 jeux	✅ Couvre l'essentiel
WhatsApp	Webhook entrant + 3 outbound channels	🟡 Proactif partiel
Sécurité	JWT, 2FA, RGPD, RLS Supabase	🟡 Quelques gaps

Roadmap globale

Sprint	Durée estimée	Statut	Thème
Sprint Correctif	~1-2j	🔴 PRIORITÉ 1	Bugs actifs bloquants
Sprint SQL	~0.5j	🟠 PRIORITÉ 2	Consolidation SQL
Sprint Tests	~2-3j	🟡 PRIORITÉ 3	Couverture tests manquants
Sprint Documentation	~1j	🟡 PRIORITÉ 4	Docs stale + manquantes
Sprint 11	~2-3j	🔵 PLANIFIÉ	Features prioritaires manquantes
Sprint 12	~2-3j	🔵 PLANIFIÉ	Architecture & Refactoring
Sprint 13	~2j	🔵 PLANIFIÉ	WhatsApp étendu + Email complet
Sprint 14	~3-4j	🔵 PLANIFIÉ	IA avancée + Cron jobs
Sprint 15	~3-4j	🔵 PLANIFIÉ	Inter-modules avancés
Sprint 16	~2j	🔵 PLANIFIÉ	Admin complet

2. Sprint Correctif — Bugs actifs

> **~1-2 jours — PRIORITÉ ABSOLUE**
> Source : `ANALYSE_COMPLETE.md` §2

🔴 C1 — Fix imports `recherche.py` *(XS)* {#-c1--fix-imports-recherchepy-*xs* }

Fichier : `src/api/routes/recherche.py` (lignes 38–44)

Problème : Imports de classes inexistantes → `ImportError` à chaque appel → barre de recherche globale 100% inutilisable.

```
# ❌ Actuel
from src.core.models import (
    Activite,    # Inexistant → ActiviteFamille
    Note,        # Inexistant → NoteMemo
    Contact,     # Inexistant → ContactFamille / ContactUtile
)

# ✅ Fix
from src.core.models import (
    ActiviteFamille,
    NoteMemo,
    ContactFamille,
)
```

Actions :

- [ ] Corriger les 3 imports dans `recherche.py`
- [ ] Mettre à jour les filtres ORM correspondants (références `Activite` → `ActiviteFamille`, etc.)
- [ ] Tester `GET /api/v1/recherche/global` en dev

🔴 C2 — Fix SQL cron `liste_courses` → `articles_courses` *(XS)* {#-c2--fix-sql-cron-liste_courses--articles_courses-*xs* }

Fichier : `src/services/core/cron/jobs.py` (ligne ~222)

Problème : La table `liste_courses` est un doublon supprimé. Le job cron `rappel_courses` (18h00) crashe silencieusement.

```
-- ❌ Actuel
SELECT COUNT(*) FROM liste_courses lc
JOIN listes_courses ls ON ls.id = lc.liste_id

-- ✅ Fix
SELECT COUNT(*) FROM articles_courses ac
JOIN listes_courses ls ON ls.id = ac.liste_id
```

Actions :

- [ ] Corriger la requête SQL dans `cron/jobs.py`
- [ ] Vérifier qu'il n'y a pas d'autres références à `liste_courses` dans les services

🟠 C3 — Fix RGPD `user_id` vide *(XS)* {#-c3--fix-rgpd-user_id-vide-*xs* }

Fichier : `src/api/routes/rgpd.py` (lignes 54, 77)

Problème : Si JWT sans `sub` ni `id`, `user_id = ""` est passé aux opérations RGPD → risque export/suppression globale.

```
# ❌ Actuel
user_id = user.get("sub", user.get("id", ""))

# ✅ Fix
user_id = user.get("sub") or user.get("id")
if not user_id:
    raise HTTPException(status_code=401, detail="Identifiant utilisateur manquant")
```

Actions :

- [ ] Corriger les 2 occurrences dans `rgpd.py` (lignes 54 et 77)

🟠 C4 — Fix mutation DB dans un GET `automations.py` *(S)* {#-c4--fix-mutation-db-dans-un-get-automationspy-*s* }

Fichier : `src/api/routes/automations.py` (lignes 38–53)

Problème : `GET /api/v1/automations` exécute un `session.commit()` si la liste est vide — violation idempotence HTTP, race conditions potentielles.

Actions :

- ☐ Extraire la logique d'initialisation dans un handler de démarrage (startup event) ou créer `POST /automations/init`
- ☐ Le `GET` ne doit faire aucune mutation

🟠 C5 — Sanitisation entrée vocale `assistant.py` *(XS)* {#-c5--sanitisation-entrée-vocale-assistantpy-*xs* }

Fichier : `src/api/routes/assistant.py` (lignes 66–67)

Problème : Commandes vocales parsées par regex et stockées sans passer par `SanitiseurDonnees` → risque injection HTML/SQL.

```
# ❌ Actuel
nom_article = course_match.group("article").strip(" .,!?")
# → ArticleCourses(nom=nom_article) stocké directement

# ✅ Fix
from src.core.validation import SanitiseurDonnees
nom_article = SanitiseurDonnees.nettoyer_texte(
    course_match.group("article").strip(" .,!?")
)
```

Actions :

- ☐ Importer `SanitiseurDonnees` dans `assistant.py`
- ☐ Appliquer `nettoyer_texte()` avant toute persistance de donnée vocale

🟡 C6 — Fix API SQLAlchemy dépréciée dans `auth.py` *(XS)* {#-c6--fix-api-sqlalchemy-dépréciée-dans-authpy-*xs* }

Fichier : `src/api/routes/auth.py` (ligne 96)

```
# ❌ Actuel (API SQLAlchemy 1.x, supprimée en 2.0)
profil = session.query(ProfilUtilisateur).get(int(user_id))

# ✅ Fix (SQLAlchemy 2.0)
profil = session.get(ProfilUtilisateur, int(user_id))
```

Actions :

- [ ] Corriger la ligne dans `auth.py`
- [ ] Scanner les autres routes pour d'autres occurrences de `.query(...).get()`

🟡 C7 — Remplacer `except Exception: pass` par logging *(M)* {#-c7--remplacer-except-exception-pass-par-logging-*m* }

12 occurrences dans :

Fichier	Lignes	Contexte
`planning.py`	L712	Météo non chargée
`planning.py`	L1123, L1150, L1169	Vue semaine incomplète
`famille.py`	L787	Budget IA données incomplètes
`famille.py`	L2683	Suggestions jardin dégradées
`dashboard.py`	L149, L281	Alertes et métriques partielles
`recherche.py`	L151, L179	Résultats vides sans raison
`export.py`	L204	Export partiel silencieux
`maison.py`	L3713	Erreur swallowed service maison

```
# ❌ Actuel
except Exception:
    pass

# ✅ Fix systématique
except Exception as e:
    logger.warning(f"[module] Erreur non bloquante: {e}")
```

Actions :

- [ ] Corriger les 12 occurrences (8 fichiers)

🟡 C8 — Exposer `push_service.obtenir_abonnes()` public *(S)* {#-c8--exposer-push_serviceobtenir_abonnes-public-s }

Fichier : `src/services/core/cron/jobs.py` (lignes 106, 133)

Problème : Accès à `push_service._subscriptions` (attribut privé). Après redémarrage, `_subscriptions` est vide → notifications ne partent pas silencieusement.

Actions :

- [ ] Ajouter méthode publique `obtenir_abonnes()` dans le service push (lecture depuis DB)
- [ ] Remplacer les 2 accès `._subscriptions.keys()` par `push_service.obtenir_abonnes()`

🟡 C9 — Auto-discovery modèles dans `backup/service.py` *(S)* {#-c9--auto-discovery-modèles-dans-backupservicepy-s }

Fichier : `src/services/core/backup/service.py` (lignes 23–47)

Problème : Liste de 25 modèles hardcodée. Les modèles `gamification.py` et `persistent_state.py` probablement non couverts (27+ fichiers modèles actuels).

```python
# ✅ Fix — auto-discovery via inspect
import inspect
from src.core import models as models_module
from src.core.db import Base

all_models = [
    cls for _, cls in inspect.getmembers(models_module, inspect.isclass)
    if issubclass(cls, Base) and cls is not Base
]
```

Actions :

- [ ] Remplacer la liste hardcodée par auto-discovery `inspect.getmembers()` sur `src.core.models`
- [ ] Valider que tous les modèles sont couverts

🔵 C10 — Masquage numéro WhatsApp dans les logs (RGPD) *(XS)* {#-c10--masquage-numéro-whatsapp-dans-les-logs-rgpd-*xs* }

Fichier : `src/services/integrations/whatsapp.py` (ligne 61)

Problème : Même tronqué à 6 chiffres, le numéro reste une donnée personnelle RGPD.

```python
# ❌ Actuel
logger.info(f"✅ Message WhatsApp envoyé à {destinataire[:6]}***")

# ✅ Fix
import hashlib
hash_dest = hashlib.sha256(destinataire.encode()).hexdigest()[:8]
logger.info(f"✅ Message WhatsApp envoyé à [hash:{hash_dest}]")
```

Actions :

- [ ] Remplacer le log par hash SHA-256 court ou `"destinataire masqué"`

🔵 C11 — Garde de rôle côté frontend page admin *(XS)* {#-c11--garde-de-rôle-côté-frontend-page-admin-*xs* }

Fichier(s) : Pages `frontend/src/app/(app)/admin/`

Problème : Absence de vérification `role === "admin"` dans les composants (defense in depth manquant côté frontend).

Actions :

- [ ] Ajouter guard `if (user?.role !== "admin") return <Redirect to="/" />` dans les pages admin concernées

3. Sprint SQL — Consolidation

> **~0.5 jour — PRIORITÉ 2**
> Source : `ANALYSE_COMPLETE.md` §4

SQL1 — Supprimer le doublon `liste_courses` *(XS)* {#sql1--supprimer-le-doublon-liste_courses-*xs* }

Fichier : `sql/INIT_COMPLET.sql`

Problème : Table `liste_courses` (singulier) est un doublon de `listes_courses`. Bug documenté B-10.

Actions :

- [ ] Supprimer le bloc `CREATE TABLE liste_courses` de `INIT_COMPLET.sql`
- [ ] Vérifier et supprimer toutes les FK qui referencent `liste_courses`
- [ ] Vérifier qu'il n'y a aucun `INSERT INTO liste_courses`

SQL2 — Ajouter les 5 index manquants *(S)* {#sql2--ajouter-les-5-index-manquants-*s* }

Fichier : `sql/INIT_COMPLET.sql`

Justification : Requêtes fréquentes sans index → full table scan.

```sql
CREATE INDEX IF NOT EXISTS idx_articles_courses_liste_achete
    ON articles_courses(liste_id, achete);

CREATE INDEX IF NOT EXISTS idx_articles_inventaire_peremption
    ON articles_inventaire(date_peremption);

CREATE INDEX IF NOT EXISTS idx_repas_planning_planning_date
    ON repas_planning(planning_id, date_repas);

CREATE INDEX IF NOT EXISTS idx_historique_actions_user_date
    ON historique_actions(user_id, created_at);

CREATE INDEX IF NOT EXISTS idx_paris_sportifs_statut_user
    ON paris_sportifs(statut, user_id);
```

Actions :

- [ ] Ajouter les 5 index dans `INIT_COMPLET.sql`

SQL3 — Vérifier et supprimer vues obsolètes Streamlit *(S)* {#sql3--vérifier-et-supprimer-vues-obsolètes-streamlit-*s* }

Fichier : `sql/INIT_COMPLET.sql`

Problème : Vues `v_autonomie`, `v_budgets_status`, `v_stats_domaine_mois` créées pour l'ancienne version Streamlit — probablement inutilisées par FastAPI.

Actions :

- [ ] Grep `v_autonomie`, `v_budgets_status`, `v_stats_domaine_mois` dans `src/api/routes/`
- [ ] Si aucun usage → supprimer les vues de `INIT_COMPLET.sql`
- [ ] Si usages → conserver et documenter

SQL4 — Ajouter trigger `listes_courses.modifie_le` *(S)* {#sql4--ajouter-trigger-listes_coursesmodifie_le-*s* }

Fichier : `sql/INIT_COMPLET.sql`

Problème : `listes_courses.modifie_le` non mis à jour quand un `articles_courses` change.

```sql
CREATE OR REPLACE FUNCTION update_liste_courses_modifie_le()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE listes_courses
    SET modifie_le = NOW()
    WHERE id = COALESCE(NEW.liste_id, OLD.liste_id);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_articles_courses_update_liste
    AFTER INSERT OR UPDATE OR DELETE ON articles_courses
    FOR EACH ROW EXECUTE FUNCTION update_liste_courses_modifie_le();
```

Actions :

- [ ] Ajouter la fonction et le trigger dans `INIT_COMPLET.sql`

SQL5 — Ajouter trigger invalidation cache planning *(S)* {#sql5--ajouter-trigger-invalidation-cache-planning-s }

Fichier : `sql/INIT_COMPLET.sql`

Problème : L'invalidation cache planning existe pour batch cooking mais pas pour les modifications directes de `repas_planning`.

Actions :

- [ ] Ajouter trigger `AFTER INSERT OR UPDATE OR DELETE ON repas_planning` → `pg_notify('planning_changed', user_id::text)`
- [ ] Vérifier côté service si un listener écoute ce canal, sinon créer le handler

4. Sprint Tests — Couverture manquante

> **~2-3 jours — PRIORITÉ 3**
> Source : `ANALYSE_COMPLETE.md` §5

T1 — `test_routes_assistant.py` *(M)* {#t1--test_routes_assistantpy-*m* }

Gap : Aucun test dédié pour `src/api/routes/assistant.py`

Actions — créer `tests/api/test_routes_assistant.py` :

- [ ] `test_commande_vocale_ajouter_article` — POST "ajoute du lait" → ArticleCourses créé
- [ ] `test_commande_vocale_sanitisee` — input avec HTML → stocké sanitisé
- [ ] `test_commande_vocale_inconnue` — commande non reconnue → réponse gracieuse
- [ ] `test_commande_vocale_sans_auth` → 401

T2 — `test_routes_whatsapp.py` *(M)* {#t2--test_routes_whatsapppy-*m* }

Gap : Aucun test pour `src/api/routes/webhooks_whatsapp.py`

Actions — créer `tests/api/test_routes_whatsapp.py` :

- [ ] `test_webhook_verification_challenge` — GET avec `hub.challenge` → retourne la valeur
- [ ] `test_webhook_message_planning` — POST texte "planning" → réponse planning
- [ ] `test_webhook_message_courses` — POST texte "courses" → réponse courses
- [ ] `test_webhook_invalid_signature` — mauvaise signature → 403
- [ ] `test_webhook_button_reply` — button_reply → machine d'état

T3 — `test_cron_jobs.py` *(M)* {#t3--test_cron_jobspy-*m* }

Gap : Aucun test pour les 19 jobs APScheduler dans `src/services/core/cron/jobs.py`

Actions — créer `tests/services/test_cron_jobs.py` :

- [ ] `test_rappels_famille_job_scheduled` — job présent dans APScheduler

- [ ] `test_rappel_courses_sql_correct` — requête utilise `articles_courses` pas `liste_courses`
- [ ] `test_alertes_peremption_48h` — articles expirés détectés correctement
- [ ] `test_resume_hebdo_structure` — dict retourné contient les sections attendues
- [ ] `test_push_contextuel_soir_notif` — mock push_service, vérifier appel
- [ ] `test_rapport_mensuel_budget` — mock email, vérifier envoi
- [ ] `test_cron_push_service_uses_public_method` — pas d'accès `._subscriptions`

T4 — `test_routes_automations.py` dédié *(S)* {#t4--test_routes_automationspy-dédié-*s* }

Gap : 6 tests partagés pour automations + voyages + garmin dans un seul fichier

Actions — créer `tests/api/test_routes_automations.py` :

- [ ] `test_lister_automations_sans_mutation` — GET ne crée rien
- [ ] `test_creer_automation_si_alors`
- [ ] `test_executer_automation_maintenant`
- [ ] `test_automation_condition_stock_bas`

T5 — `test_routes_voyages.py` + `test_routes_garmin.py` *(S)* {#t5--test_routes_voyagespy--test_routes_garminpy-*s* }

Actions — créer les 2 fichiers :

- [ ] `tests/api/test_routes_voyages.py` : CRUD voyages, génération courses, événement `voyage.en_cours`
- [ ] `tests/api/test_routes_garmin.py` : sync matinal, recommandation dîner, score bien-être

T6 — Tests services d'intégration sans couverture *(M)* {#t6--tests-services-dintégration-sans-couverture-*m* }

Gap : 5 services d'intégration sans aucun test

Actions :

- [ ] `tests/services/test_whatsapp_service.py` : envoi message, envoi interactif, masquage numéro
- [ ] `tests/services/test_google_calendar.py` : sync planning, création événement (mock OAuth)
- [ ] `tests/services/test_ticket_caisse.py` : parsing ticket → articles dépenses
- [ ] `tests/services/test_facture.py` : parsing facture → données structurées
- [ ] `tests/services/test_recettes_enrichers.py` : enrichissement recette → données complètes

T7 — Tests famille Phase 6 *(M)* {#t7--tests-famille-phase-6-*m* }

Gap : Tests prévus mais non créés

Actions :

- [ ] Créer `tests/api/test_famille_garde.py` : config zones, semaines fermeture, jours-sans-crèche
- [ ] Compléter `tests/api/test_famille_achats.py` : CRUD + suggestions IA (mock Mistral) + scoring + Vinted

T8 — Fichiers `error.tsx` et `loading.tsx` manquants *(S)* {#t8--fichiers-errortsx-et-loadingtsx-manquants-*s* }

Gap : Pages admin, ma-semaine et dashboard racine sans gestion d'erreur Next.js

Actions :

- [ ] Créer `frontend/src/app/(app)/admin/error.tsx`
- [ ] Créer `frontend/src/app/(app)/admin/loading.tsx`
- [ ] Créer `frontend/src/app/(app)/ma-semaine/error.tsx`
- [ ] Créer `frontend/src/app/(app)/ma-semaine/loading.tsx`
- [ ] Créer `frontend/src/app/(app)/error.tsx` (root app)

T9 — Tests pages frontend manquantes *(M)* {#t9--tests-pages-frontend-manquantes-*m* }

Gap : Pages récentes sans test Vitest

Actions — créer dans `frontend/src/__tests__/` :

- [ ] `admin/jobs.test.tsx`
- [ ] `cuisine/photo-frigo.test.tsx`
- [ ] `famille/timeline.test.tsx`
- [ ] `outils/assistant-vocal.test.tsx`
- [ ] `outils/nutritionniste.test.tsx`

5. Sprint Documentation

> **~1 jour — PRIORITÉ 4**
> Source : `ANALYSE_COMPLETE.md` §6

D1 — Mettre à jour `docs/ARCHITECTURE.md` *(S)* {#d1--mettre-à-jour-docsarchitecturemd-s }

Problème : Modules Garmin, automations, voyages et gamification non documentés dans ARCHITECTURE.md.

Actions :

- [ ] Ajouter section "Intégration Garmin" dans ARCHITECTURE.md
- [ ] Ajouter section "Module Gamification" dans ARCHITECTURE.md
- [ ] Ajouter section "Module Voyages" dans ARCHITECTURE.md
- [ ] Ajouter section "Automations Si→Alors" dans ARCHITECTURE.md
- [ ] Mettre à jour le schéma architecture globale (WebSocket x4, 44+ routers)
- [ ] Mettre à jour la liste des services (80+ services)

D2 — Mettre à jour `docs/API_REFERENCE.md` *(S)* {#d2--mettre-à-jour-docsapi_referencemd-s }

Problème : Routes manquantes : Garmin, automations, voyages, gamification.

Actions :

- [ ] Ajouter les endpoints manquants : `/garmin/*` , `/automations/*` , `/famille/voyages/*` , `/famille/gamification/*`
- [ ] Vérifier cohérence avec les schémas Pydantic actuels

D3 — Mettre à jour `docs/SERVICES_REFERENCE.md` *(S)* {#d3--mettre-à-jour-docsservices_referencemd-*s* }

Problème : Famille refonte non reflétée, nouveaux services (Garmin, voyages, gamification, automations) absents.

Actions :

- ☐ Documenter la séparation famille → jules, budget, activités, garde
- ☐ Ajouter les services Garmin, gamification, voyage, automations

D4 — Régénérer `docs/ERD_SCHEMA.md` *(M)* {#d4--régénérer-docserd_schemamd-*m* }

Problème : Environ 100 tables documentées vs 130 réelles. V002 non reflétée.

Actions :

- ☐ Recenser les 130 tables actuelles depuis `sql/INIT_COMPLET.sql`
- ☐ Regrouper par domaine (cuisine, famille, maison, jeux, planning, admin, intégrations)
- ☐ Mettre à jour le diagramme ERD + les relations FK principales

D5 — Supprimer `docs/REDIS_SETUP.md` → créer `docs/CACHE_SETUP.md` *(S)* {#d5--supprimer-docsredis_setupmd--créer-docscache_setupmd-*s* }

Problème : `REDIS_SETUP.md` est trompeur — Redis n'est pas utilisé. L'app utilise cache L1 mémoire + L3 fichier (+ L2 Redis optionnel).

Actions :

- ☐ Supprimer `docs/REDIS_SETUP.md`
- ☐ Créer `docs/CACHE_SETUP.md` : L1 mémoire (TTL court), L2 Redis (optionnel), L3 fichier (TTL long), `@avec_cache`, invalidation
- ☐ Mettre à jour `docs/INDEX.md` (retirer REDIS, ajouter CACHE)

D6 — Créer `docs/WHATSAPP.md` *(S)* {#d6--créer-docswhatsappmd-*s* }

Actions :

- [ ] Machine d'état conversationnelle (commandes, states, transitions)
- [ ] Setup Meta Cloud API (variables env requis, webhook URL, verification token)
- [ ] Canaux sortants : messages simples, interactifs, templates
- [ ] Liste commandes actuelles : `planning` , `courses` , `frigo` , `menu`
- [ ] Liste commandes à venir : `jules` , `ajouter` , `budget` , `anniversaires` , `recette` , `tâches`

D7 — Créer `docs/CRON_JOBS.md` *(S)* {#d7--créer-docscron_jobsmd-*s* }

Actions :

- [ ] Tableau complet des 19 jobs APScheduler + 6 jeux : ID, schedule, description, canaux, dépendances
- [ ] Section "Jobs à créer" (6 jobs proposés dans l'analyse)
- [ ] Section "Monitoring" : logs, détection échec, trigger manuel
- [ ] Section "Trigger manuel" via endpoint admin

D8 — Créer `docs/ADMIN_GUIDE.md` *(S)* {#d8--créer-docsadmin_guidemd-*s* }

Actions :

- [ ] Mode admin : accès, rôles, navigation
- [ ] Audit logs : format, retention, export CSV/JSON
- [ ] Backup : commandes CLI, fréquence
- [ ] Trigger manuel cron jobs depuis UI
- [ ] Gestion utilisateurs (liste, désactivation, reset password)
- [ ] Test notifications (WhatsApp, push, email)
- [ ] Cache stats + purge

D9 — Créer `docs/AUTOMATIONS.md` *(S)* {#d9--créer-docsautomationsmd-*s* }

Actions :

- [ ] Architecture moteur Si→Alors
- [ ] Conditions disponibles : `stock_bas` , `peremption_proche` , `meteo_xxx` , `budget_depasse`
- [ ] Actions disponibles : `ajouter_courses` , `notifier` , `creer_tache` , `desactiver_planning`
- [ ] Format JSON des règles
- [ ] Exemples de règles pratiques

D10 — Créer `docs/GARMIN.md` *(S)* {#d10--créer-docsgarminmd-*s* }

Actions :

- [ ] OAuth 2.0 setup Garmin Connect API
- [ ] Scopes requis (activités, santé, sommeil)
- [ ] Sync matinal : données importées, modèles DB (`activites_garmin` , `calories_actives_garmin`)
- [ ] Endpoint recommandation dîner selon calories brûlées
- [ ] Contribution au score bien-être

D11 — Mettre à jour `ROADMAP.md` *(S)* {#d11--mettre-à-jour-roadmapmd-*s* }

Problème : Sprint 8-9 marqués "À FAIRE" alors que déjà partiellement livrés.

Actions :

- [ ] Fermer les items Sprint 8 et Sprint 9 déjà livrés
- [ ] Aligner sur l'état réel actuel
- [ ] Ajouter sprints correctifs, SQL, tests, docs comme prochaines priorités

6. Sprint 11 — Features manquantes prioritaires

> **~2-3 jours**
> Source : `ANALYSE_COMPLETE.md` §3 (Gaps par module)

🟠 F3 — Rappels jour-par-jour (notifications planning) *(M)* {#-f3--rappels-jour-par-jour-notifications-planning-*m* }

État : Manquant.

Actions :

- [ ] Ajouter dans `rappels_generaux` une vérification `repas_planning` du jour
- [ ] Notifier via ntfy + push : "Ce soir : [recette]" avec les ingrédients à sortir
- [ ] Paramètre opt-in dans les préférences utilisateur

🟠 F4 — Logs sécurité admin *(M)* {#-f4--logs-sécurité-admin-*m* }

État : Manquant.

Actions :

- [ ] Créer table `logs_securite` : `user_id` , `event_type` , `ip` , `user_agent` , `created_at` (SQL + ORM)
- [ ] Ajouter RLS sur la table (admin only)
- [ ] Logger les événements dans les middlewares rate_limiting et auth
- [ ] Endpoint `GET /api/v1/admin/security-logs` avec filtres type/date
- [ ] Widget dans la page admin avec les derniers événements suspects

🟡 F5 — Pages frontend maison manquantes *(S)* {#-f5--pages-frontend-maison-manquantes-*s* }

État : API existante, pages absentes.

Actions :

- [ ] Créer `frontend/src/app/(app)/maison/contrats/page.tsx`

- [ ] Créer `frontend/src/app/(app)/maison/artisans/page.tsx`
- [ ] Créer `frontend/src/app/(app)/maison/diagnostics/page.tsx`
- [ ] Ajouter les liens dans la navigation maison

🟡 F6 — Connecter scan ticket caisse → dépenses auto *(S)* {#-f6--connecter-scan-ticket-caisse--dépenses-auto-s }

État : `src/services/integrations/ticket_caisse.py` existe mais non connecté.

Actions :

- [ ] Endpoint `POST /api/v1/maison/depenses/import-ticket` (upload photo + parsing OCR)
- [ ] Afficher résultat parsé avec confirmation avant import
- [ ] Bouton "Importer depuis photo" dans la page dépenses

🟡 F7 — Export carnet de santé Jules PDF *(S)* {#-f7--export-carnet-de-santé-jules-pdf-s }

État : Service PDF existe, pas d'endpoint dédié.

Actions :

- [ ] Créer endpoint `GET /api/v1/famille/jules/carnet-sante/pdf`
- [ ] Générer PDF : courbes croissance, jalons, vaccins, aliments exclus, coaching IA
- [ ] Bouton "Télécharger le carnet de santé" dans la page Jules

🟡 F8 — Partage recette (lien public + PDF) *(S)* {#-f8--partage-recette-lien-public--pdf-s }

État : Entièrement manquant.

Actions :

- [ ] Endpoint `POST /api/v1/recettes/{id}/partager` → génère token temporaire (24-48h)
- [ ] Route publique `GET /share/recette/{token}` (sans auth)
- [ ] Endpoint `GET /api/v1/recettes/{id}/export-pdf`

- [ ] Bouton "Partager" dans la fiche recette

🟡 F9 — Notation recette affichée dans la liste *(XS)* {#-f9--notation-recette-affichée-dans-la-liste-*xs* }

État : Endpoint `feedback` existe, non affiché dans la liste.

Actions :

- [ ] Dans `frontend/src/app/(app)/cuisine/recettes/page.tsx` → afficher icônes feedback dans les cards
- [ ] Filtre par note dans les options de tri

🟡 F10 — Gestion conflits cross-module visible dans l'UI *(S)* {#-f10--gestion-conflits-cross-module-visible-dans-lui-*s* }

État : `ServiceConflits.detecter_conflits()` existe, non surfacé.

Actions :

- [ ] Endpoint `GET /api/v1/planning/conflits`
- [ ] Widget "Conflits" dans la vue planning (ex: "Activité Jules × repas 19h00")
- [ ] Option de résolution rapide (déplacer l'un ou l'autre)

🔵 F11 — Vue calendrier mensuel planning *(M)* {#-f11--vue-calendrier-mensuel-planning-*m* }

État : Vue hebdomadaire seulement.

Actions :

- [ ] Endpoint `GET /api/v1/planning/mensuel?mois=YYYY-MM`
- [ ] Composant `CalendrierMensuel` dans `frontend/src/composants/`
- [ ] Toggle semaine/mois dans la page planning

🔵 F12 — Export planning PDF depuis l'UI *(XS)* {#-f12--export-planning-pdf-depuis-lui-*xs* }

État : Endpoint export existe, pas de bouton dans la page planning.

Actions :

- [ ] Ajouter bouton "Exporter en PDF" dans `frontend/src/app/(app)/planning/page.tsx`
- [ ] Appel vers `GET /api/v1/export/planning` avec paramètres semaine/mois

🔵 F13 — Interface backtest stratégies paris *(S)* {#-f13--interface-backtest-stratégies-paris-*s* }

État : Service existe, UI partielle.

Actions :

- [ ] Compléter `frontend/src/app/(app)/jeux/paris/page.tsx` avec onglet "Backtests"
- [ ] Graphique performance stratégie vs bankroll dans le temps

🔵 F14 — Détection patterns jeux (hot hand / régression) *(S)* {#-f14--détection-patterns-jeux-hot-hand--régression-*s* }

État : TODO dans `jeux.py` ligne ~372.

Actions :

- [ ] Implémenter l'analyse statistique des séries de paris
- [ ] Endpoint `GET /api/v1/jeux/patterns` → détection séquences chaudes/froides
- [ ] Widget dans la page paris : "Attention — 5 paris perdants consécutifs"

7. Sprint 12 — Architecture & Refactoring

> **~2-3 jours**
>
> Source : `ANALYSE_COMPLETE.md` §13

A1 — Splitter `maison.py` (~3700 lignes) *(L)* {#a1--splitter-maisonpy-3700-lignes-*l* }

Problème : Fichier trop grand, maintenabilité dégradée.

Actions :

- [ ] Créer `src/api/routes/maison_projets.py`
- [ ] Créer `src/api/routes/maison_entretien.py`
- [ ] Créer `src/api/routes/maison_finances.py` (dépenses, contrats, artisans)
- [ ] Créer `src/api/routes/maison_jardin.py`
- [ ] Garder `maison.py` comme router central avec `include_router`
- [ ] Mettre à jour `src/api/routes/__init__.py` et `main.py`
- [ ] Vérifier qu'aucun test ne casse

A2 — Splitter `famille.py` (~2700 lignes) *(L)* {#a2--splitter-famillepy-2700-lignes-*l* }

Actions :

- [ ] Créer `src/api/routes/famille_jules.py`
- [ ] Créer `src/api/routes/famille_budget.py`
- [ ] Créer `src/api/routes/famille_activites.py`
- [ ] Garder `famille.py` comme re-export/router central
- [ ] Mettre à jour `__init__.py` et `main.py`

A3 — Uniformiser noms factory (tout en français) *(S)* {#a3--uniformiser-noms-factory-tout-en-français-*s* }

Problème : Mélange `get_xxx_service()` (anglais) et `obtenir_service_xxx()` (français).

Actions :

- [ ] Lister toutes les fonctions factory `get_*_service` dans `src/services/`
- [ ] Renommer en `obtenir_*_service()` par module
- [ ] Mettre à jour les imports dans les routes correspondantes
- [ ] Mettre à jour les tests

A4 — Génération automatique types TypeScript depuis Pydantic *(M)* {#a4--génération-automatique-types-typescript-depuis-pydantic-*m* }

Problème : Types `frontend/src/types/` maintenus manuellement → risque de désynchronisation.

Actions :

- ☐ Installer `openapi-typescript` : `npm install -D openapi-typescript`
- ☐ Configurer script `npm run generate-types` →
  `openapi-typescript http://localhost:8000/openapi.json -o src/types/api-generated.ts`
- ☐ Ajouter au `package.json`
- ☐ Documenter dans le README frontend

A5 — Audit tables orphelines ORM ↔ SQL *(M)* {#a5--audit-tables-orphelines-orm--sql-*m* }

Source : `ANALYSE_COMPLETE.md` §4

Actions :

- ☐ Lister les `__tablename__` de `src/core/models/voyage.py` → vérifier dans INIT_COMPLET
- ☐ Vérifier `journal_bord`, `sessions_travail`, `log_statut_objets` : ORM existant ?
- ☐ Vérifier `gamification.py` et `persistent_state.py` : présents en SQL ?
- ☐ Pour orphelines SQL sans ORM : créer le modèle ou supprimer la table
- ☐ Pour ORM sans SQL : ajouter CREATE TABLE ou supprimer le modèle

8. Sprint 13 — WhatsApp étendu + Email complet

> **~2 jours**
> Source : `ANALYSE_COMPLETE.md` §11

W1 — Compléter TODO IA régénération planning WhatsApp *(S)* {#w1--compléter-todo-ia-régénération-planning-whatsapp-*s* }

Fichier : `src/api/routes/webhooks_whatsapp.py` (TODO ligne 131)

Actions :

- ☐ Implémenter le handler `regenerer_planning_ia()` sur button reply "Régénérer"
- ☐ Appeler `PlanningService.generer_planning_ia()` et envoyer le nouveau planning en WhatsApp

W2 — Nouvelles commandes WhatsApp *(M)* {#w2--nouvelles-commandes-whatsapp-*m* }

Actions — ajouter dans `webhooks_whatsapp.py` :

Commande	Action	Effort
`jules` / `bébé`	Résumé Jules : repas, activités, jalons récents	S
`ajouter [article]`	Ajouter directement en liste courses active	S
`budget`	Budget mensuel en cours vs objectif	S
`anniversaires`	Prochains anniversaires 30 jours	XS
`recette [nom]`	Fiche recette avec ingrédients	S
`tâches`	Tâches maison en retard	S
`aide admin`	(admin only) liste commandes admin	XS

- ☐ Ajouter les 7 commandes dans la machine d'état WhatsApp

W3 — Compléter dispatcher email SMTP *(M)* {#w3--compléter-dispatcher-email-smtp-*m* }

État : `ServiceEmail` via Resend configuré, certains envois non câblés.

Actions :

- [ ] Rapport hebdo famille → vérifier que l'email est envoyé (cron `resume_hebdo`)
- [ ] Rapport mensuel budget → email en plus de ntfy
- [ ] Alertes critiques → email si péremption < 24h OU garantie < 30j
- [ ] Tester en dev avec compte Resend sandbox

W4 — Préférences canaux notification utilisateur *(S)* {#w4--préférences-canaux-notification-utilisateur-*s* }

Actions :

- [ ] Ajouter champ `preferences_notifications` JSONB dans `preferences_utilisateurs` (SQL + ORM)
- [ ] Endpoint `GET/PUT /api/v1/preferences/notifications`
- [ ] UI dans les paramètres : toggle par canal (ntfy, WhatsApp, email, push) par catégorie (rappels, alertes, résumés)

9. Sprint 14 — Nouvelles opportunités IA + Cron jobs manquants

> **~3-4 jours**
> Source : `ANALYSE_COMPLETE.md` §9 + §10

IA1 — Résumé famille contextualisé hebdomadaire *(M)* {#ia1--résumé-famille-contextualisé-hebdomadaire-*m* }

Valeur : ⭐ ⭐ ⭐ Haute

Actions :

- [ ] `src/services/dashboard/resume_famille_ia.py` — résumé bienveillant via Mistral
- [ ] Contexte injecté : planning semaine + inventaire + budget + score Jules + météo
- [ ] Endpoint `GET /api/v1/dashboard/resume-hebdo-ia`
- [ ] Widget dashboard "Votre semaine en un coup d'œil"
- [ ] Inclure dans le cron `resume_hebdo` (envoi push + email vendredi soir)

IA2 — Prédiction liste courses *(L)* {#ia2--prédiction-liste-courses-*l* }

Valeur : ⭐⭐⭐ Haute — **Prérequis :** Historique achats > 4 semaines

Actions :

- [ ] `src/services/cuisine/prediction_courses.py` — analyse historique `articles_courses` + fréquence
- [ ] Endpoint `GET /api/v1/courses/predictions` → liste pré-complétée avec scores confiance
- [ ] Dans la page courses : section "Articles habituels" (checkboxes pré-cochées)
- [ ] Amélioration progressive via feedback (ajout/refus)

IA3 — Détection anomalies dépenses *(M)* {#ia3--détection-anomalies-dépenses-*m* }

Valeur : ⭐⭐⭐ Haute

Actions :

- [ ] `src/services/dashboard/anomalies_financieres.py` — compare mois courant vs N-1, N-2
- [ ] Endpoint `GET /api/v1/dashboard/anomalies-financieres`
- [ ] Alertes : +X% vs moyenne habituelle par catégorie (courses, énergie, loisirs)
- [ ] Widget dashboard avec recommandations IA

IA4 — Chat IA contextuel cross-module enrichi *(M)* {#ia4--chat-ia-contextuel-cross-module-enrichi-*m* }

État : Chat IA existe, contexte limité.

Actions :

- [ ] Injecter dans le prompt système : planning + inventaire + budget + score Jules + événements famille
- [ ] Mémoire conversationnelle courte (5 derniers échanges)
- [ ] Endpoint `POST /api/v1/assistant/chat` avec contexte enrichi

IA5 — Suggestions activités Jules contextuelles *(S)* {#ia5--suggestions-activités-jules-contextuelles-*s* }

Valeur : ⭐ ⭐

Actions :

- [ ] Dans `JulesAIService` : enrichir le prompt avec météo + crèche ouverte/fermée + âge Jules
- [ ] Endpoint `GET /api/v1/famille/jules/activites-suggestions?contexte=meteo_pluie`
- [ ] Widget dans la page famille Jules

IA6 — Génération automatique d'automations via IA *(M)* {#ia6--génération-automatique-dautomations-via-ia-*m* }

Valeur : ⭐ ⭐

Actions :

- [ ] Endpoint `POST /api/v1/automations/generer-ia` — prompt libre → règle Si→Alors JSON
- [ ] Mistral génère `{ condition, action, parametres }`
- [ ] Interface : champ texte libre → prévisualisation règle → confirmation

IA7 — Prédiction péremption personnalisée *(S)* {#ia7--prédiction-péremption-personnalisée-*s* }

Valeur : ⭐ ⭐

Actions :

- [ ] Analyser durée de vie réelle des produits (date achat → date consommation)
- [ ] Modèle simple : durée_vie_moyenne par catégorie × facteur conservation
- [ ] Alertes proactives basées sur ce modèle plutôt que date DB fixe

J1 — Job `sync_google_calendar` *(S)* {#j1--job-sync_google_calendar-s }

Attribut	Valeur
Schedule	Quotidien 23:00
Description	Sync planning repas + activités → Google Calendar
Canal	interne
Dépendance	`google_calendar.py` service existant

Actions :

- ☐ Ajouter job APScheduler `sync_google_calendar` (23h00 quotidien)
- ☐ Sync des events créés/modifiés depuis 24h

J2 — Job `rapport_nutrition_jules` *(S)* {#j2--job-rapport_nutrition_jules-s }

Attribut	Valeur
Schedule	Dimanche 19:00
Description	Bilan nutrition Jules semaine
Canal	ntfy + email

Actions :

- ☐ Agréger repas Jules de la semaine depuis `repas_planning` (champ `plat_jules`)
- ☐ Calculer score diversité, aliments nouveaux, aliments refusés
- ☐ Envoyer résumé ntfy + email dimanche soir

J3 — Job `alerte_stock_bas` *(S)* {#j3--job-alerte_stock_bas-*s* }

Attribut	Valeur
Schedule	Quotidien 07:00
Description	Articles inventaire < seuil minimum → ajout auto liste courses
Canal	ntfy

Actions :

- ☐ Requête `articles_inventaire WHERE quantite < seuil_minimum AND actif = true`
- ☐ Créer ou compléter la liste courses du jour
- ☐ Notification ntfy "X articles ajoutés automatiquement"

J4 — Job `archive_batches_expires` *(XS)* {#j4--job-archive_batches_expires-*xs* }

Attribut	Valeur
Schedule	Quotidien 02:00
Description	Archiver préparations batch expirées

Actions :

- ☐ `UPDATE preparations_batch SET archive = true WHERE date_expiration < NOW()`

J5 — Job `rapport_maison_mensuel` *(S)* {#j5--job-rapport_maison_mensuel-*s* }

Attribut	Valeur
Schedule	1er du mois 09:30
Description	État maison : projets actifs, entretiens à venir, dépenses mois
Canal	ntfy + email

Actions :

- [ ] Agréger : projets en cours, entretiens planifiés N+30j, dépenses maison mois N-1
- [ ] Envoyer ntfy + email

J6 — Job `sync_openFoodFacts` *(S)* {#j6--job-sync_openfoodfacts-*s* }

Attribut	Valeur
Schedule	Dimanche 03:00
Description	Refresh cache OpenFoodFacts pour les articles fréquents

Actions :

- [ ] Identifier les produits scannés les 30 derniers jours
- [ ] Rafraîchir les données nutritionnelles depuis OpenFoodFacts API
- [ ] Mettre à jour `articles_inventaire.donnees_nutritionnelles`

10. Sprint 15 — Interactions inter-modules avancées

> **~3-4 jours**
> Source : `ANALYSE_COMPLETE.md` §7 + §8

IM1 — Cuisine × Famille : adaptation auto repas adultes → Jules (semaine entière) *(M)* {#im1--cuisine--famille--adaptation-auto-repas-adultes--jules-semaine-entière-*m* }

Valeur : Gain de temps majeur

Actions :

- [ ] `ServiceVersionRecetteJules.adapter_planning(planning_id)` → batch sur tous les repas de la semaine
- [ ] Endpoint `POST /api/v1/planning/{id}/adapter-jules`
- [ ] UI : bouton "Adapter toute la semaine pour Jules" dans le planning

IM2 — Maison (Jardin) × Cuisine : stock récoltes → recettes suggérées *(S)* {#im2--maison-jardin--cuisine--stock-récoltes--recettes-suggérées-*s* }

Valeur : Cohérence local/bio

Actions :

- [ ] Endpoint `GET /api/v1/recettes/depuis-jardin` → récoltes disponibles → recettes suggérées
- [ ] Mention dans le rapport jardin hebdo (cron `rapport_jardin`)
- [ ] Widget dashboard cuisine "Récolter et cuisiner"

IM3 — Jeux × Famille (Budget) : alerte paris > seuil semaine *(S)* {#im3--jeux--famille-budget--alerte-paris--seuil-semaine-*s* }

Valeur : Gaming responsable

Actions :

- [ ] Paramètre `seuil_paris_semaine` dans les préférences utilisateur
- [ ] Dans `BudgetGuardMiddleware` : vérifier cumul paris semaine courante
- [ ] Si dépassement : notif ntfy + WhatsApp + entrée `logs_securite`
- [ ] Dashboard jeux : indicateur "Budget paris : X€ / Y€ cette semaine"

IM4 — Planning × Famille (Voyages) : pause automatique planning lors voyage *(S)* {#im4--planning--famille-voyages--pause-automatique-planning-lors-voyage-*s* }

État : Événement `voyage.en_cours` émis mais non intercepté par le planning.

Actions :

- [ ] Dans le service planning : écouter `voyage.en_cours` via Event Bus
- [ ] Suspendre la génération automatique du planning pendant le voyage
- [ ] Reprendre à la date de retour
- [ ] Notification "Planning mis en pause — voyage détecté"

IM5 — Maison (Energie) × Cuisine : heures creuses → suggérer appareils *(S)* {#im5--maison-energie--cuisine--heures-creuses--suggérer-appareils-*s* }

Valeur : Optimisation éco/coût

Actions :

- [ ] Enrichir les données énergie avec les plages heures creuses (configurable)
- [ ] Endpoint `GET /api/v1/cuisine/suggestions-heures-creuses`
- [ ] Widget cuisine : "Heures creuses de 22h à 6h — lancez le Cookeo maintenant"

IM6 — Garmin × Planning (Nutrition) : calories brûlées → ajustement macros *(M)* {#im6--garmin--planning-nutrition--calories-brûlées--ajustement-macros-*m* }

Valeur : Santé personnalisée

Actions :

- [ ] Dans `garmin_sync_matinal` : calculer les macros supplémentaires nécessaires
- [ ] Endpoint `GET /api/v1/planning/ajustement-macros-garmin`
- [ ] Affichage dans le planning : "Vous avez brûlé +450 kcal → ajustez le dîner"

IM7 — Garmin × Famille (Bien-être) : dashboard santé famille *(M)* {#im7--garmin--famille-bien-être--dashboard-santé-famille-*m* }

Valeur : Vue globale santé famille

Actions :

- [ ] Endpoint `GET /api/v1/famille/sante-globale` — score Jules + données Garmin adultes
- [ ] Widget dashboard "Santé famille cette semaine"
- [ ] Métriques : activité adultes (Garmin), sommeil, nutrition Jules, score bien-être global

IM8 — Jeux × Outils (Automations) : gain jeux → journal/budget *(S)* {#im8--jeux--outils-automations--gain-jeux--journalbudget-*s* }

Actions :

- [ ] Action automation `noter_gain_jeux` — si gain > X€ → entrée journal + ligne budget
- [ ] Exemple règle : "Si gain paris > 50€ → noter dans journal + ajouter au budget"

IM9 — Maison (Diagnostics IA) × Artisans : workflow réparation complet *(M)* {#im9--maison-diagnostics-ia--artisans--workflow-réparation-complet-*m* }

Valeur : Photo → diagnostic → artisans en 3 étapes

Actions :

- [ ] `POST /api/v1/maison/diagnostics/ia-photo` → Pixtral → identification panne + estimation coût
- [ ] Résultat diag → lien vers liste artisans filtrée par type de panne
- [ ] Création automatique d'un projet maison depuis le diagnostic
- [ ] Page `maison/diagnostics/page.tsx` (voir Sprint 11 F5)

11. Sprint 16 — Admin complet

> **~2 jours**
>
> Source : `ANALYSE_COMPLETE.md` §12

ADM1 — Endpoints admin manquants *(M)* {#adm1--endpoints-admin-manquants-*m* }

Actions — créer/compléter dans `src/api/routes/admin.py` :

```
POST /api/v1/admin/jobs/{job_id}/run      → Trigger manuel d'un cron job
GET  /api/v1/admin/jobs                   → Liste + dernière exécution + statut
GET  /api/v1/admin/jobs/{job_id}/logs    → Logs dernière exécution
POST /api/v1/admin/whatsapp/test          → Message WhatsApp de test
POST /api/v1/admin/push/test              → Push notification de test
POST /api/v1/admin/email/test             → Email de test
GET  /api/v1/admin/services/health        → Health check registre services
GET  /api/v1/admin/cache/stats            → Statistiques cache hit/miss/size
POST /api/v1/admin/cache/clear            → Vider cache L1 + L3
GET  /api/v1/admin/users                  → Liste utilisateurs
POST /api/v1/admin/users/{id}/disable     → Désactiver compte
GET  /api/v1/admin/db/coherence           → Lancer test cohérence rapide
```

- [ ] Tous avec `Depends(require_role("admin"))`
- [ ] Rate limiting spécifique admin : 5 req/min sur triggers jobs
- [ ] Audit log automatique sur chaque action admin

ADM2 — Pages frontend admin manquantes *(M)* {#adm2--pages-frontend-admin-manquantes-*m* }

```
frontend/src/app/(app)/admin/
  ├── page.tsx                    → Audit logs (existant ✅)
  ├── jobs/page.tsx               → À compléter : liste + trigger + logs
  ├── services/page.tsx           → À créer : health check + cache stats
  ├── notifications/page.tsx      → À créer : test WhatsApp/Push/Email + historique
  └── utilisateurs/page.tsx       → À créer : liste users + désactiver + reset
```

Actions :

- [ ] Compléter `admin/jobs/page.tsx` : table jobs, bouton ▶ Exécuter, toast feedback, logs
- [ ] Créer `admin/services/page.tsx` : health services + stats cache
- [ ] Créer `admin/notifications/page.tsx` : tester canaux + historique envois
- [ ] Créer `admin/utilisateurs/page.tsx` : liste users + actions (désactiver, reset password)

ADM3 — Sécurité audit logs généralisée *(S)* {#adm3--sécurité-audit-logs-généralisée-*s* }

Actions :

- [ ] Généraliser l'audit log automatique à toutes les actions admin
- [ ] Intégrer logs sécurité (Sprint 11 F4) dans la page admin (`admin/page.tsx`)
- [ ] Navigation admin conditionnelle `role === "admin"` vérifiée dans chaque composant (defense in depth)

12. Dépendances entre items

```
C1 (fix recherche imports) ──────────→ T1 (tests assistant)
C2 (fix SQL cron) ──────────────────→ T3 (tests cron jobs)
SQL1 (supprimer liste_courses) ──────→ C2 (à faire en même temps)
SQL2 (index) ────────────────────────→ meilleures perfs pour IA3 (anomalies)
C8 (méthode publique push) ──────────→ T3 (tests cron — accès public)
T6 (tests services intégrations) ───→ W1 (WhatsApp IA complet)
D5 (supprimer REDIS_SETUP.md) ───────→ docs cohérentes
D7 (CRON_JOBS.md) ───────────────────→ J1 à J6 (référence pour nouveaux jobs)
F2 (profil diététique) ──────────────→ IA3 (anomalies dépenses nutrition) + F3 (rappels)
W4 (préférences canaux) ─────────────→ F3 (rappels planning opt-in)
IA1 (résumé hebdo IA) ───────────────→ W2 (commande WhatsApp `résumé`)
IA2 (prédiction courses) ────────────→ J3 (alerte stock bas)
IM6 (Garmin × nutrition) ────────────→ J2 (rapport nutrition Jules)
A1 (split maison.py) ────────────────→ ADM1 (endpoints admin lisibles)
A2 (split famille.py) ───────────────→ T7 (tests famille plus ciblés)
A3 (unifier factory) ────────────────→ A4 (types TS proprement générés)
ADM1 (endpoints admin) ──────────────→ ADM2 (pages frontend admin)
F4 (logs sécurité) ──────────────────→ ADM3 (admin audit généralisé)
F5 (pages maison) ───────────────────→ IM9 (diagnostics IA × artisans)
```

13. Suivi d'avancement global

Sprint Correctif

- ☐ C1 — Fix imports `recherche.py` (ActiviteFamille, NoteMemo, ContactFamille)
- ☐ C2 — Fix SQL cron `liste_courses` → `articles_courses`
- ☐ C3 — Fix RGPD user_id vide
- ☐ C4 — Fix mutation DB dans GET automations
- ☐ C5 — Sanitisation entrée vocale assistant
- ☐ C6 — Fix API SQLAlchemy dépréciée auth.py
- ☐ C7 — Remplacer 12x `except: pass` par logging
- ☐ C8 — Méthode publique `push_service.obtenir_abonnes()`
- ☐ C9 — Auto-discovery modèles backup/service.py
- ☐ C10 — Masquage numéro WhatsApp logs RGPD
- ☐ C11 — Garde de rôle frontend pages admin

Sprint SQL

- ☐ SQL1 — Supprimer doublon `liste_courses`
- ☐ SQL2 — 5 index manquants
- ☐ SQL3 — Vérifier/supprimer vues Streamlit obsolètes
- ☐ SQL4 — Trigger `listes_courses.modifie_le`
- ☐ SQL5 — Trigger invalidation cache planning

Sprint Tests

- ☐ T1 — `test_routes_assistant.py`
- ☐ T2 — `test_routes_whatsapp.py`
- ☐ T3 — `test_cron_jobs.py`
- ☐ T4 — `test_routes_automations.py` dédié
- ☐ T5 — `test_routes_voyages.py` + `test_routes_garmin.py`
- ☐ T6 — Tests services intégrations (5 fichiers)
- ☐ T7 — Tests famille Phase 6 (garde + achats)
- ☐ T8 — `error.tsx` + `loading.tsx` manquants (admin, ma-semaine, root)
- ☐ T9 — Tests pages frontend (5 fichiers)

Sprint Documentation

- [ ] D1 — Mettre à jour `ARCHITECTURE.md`
- [ ] D2 — Mettre à jour `API_REFERENCE.md`
- [ ] D3 — Mettre à jour `SERVICES_REFERENCE.md` (famille refonte)
- [ ] D4 — Régénérer `ERD_SCHEMA.md` (130 tables)
- [ ] D5 — Supprimer `REDIS_SETUP.md` → créer `CACHE_SETUP.md`
- [ ] D6 — Créer `docs/WHATSAPP.md`
- [ ] D7 — Créer `docs/CRON_JOBS.md`
- [ ] D8 — Créer `docs/ADMIN_GUIDE.md`
- [ ] D9 — Créer `docs/AUTOMATIONS.md`
- [ ] D10 — Créer `docs/GARMIN.md`
- [ ] D11 — Mettre à jour `ROADMAP.md`

Sprint 11 — Features prioritaires

- [ ] F3 — Rappels jour-par-jour notifications planning
- [ ] F4 — Logs sécurité admin
- [ ] F5 — Pages frontend maison (contrats, artisans, diagnostics)
- [ ] F6 — Scan ticket caisse → dépenses auto
- [ ] F7 — Export carnet de santé Jules PDF
- [ ] F8 — Partage recette (lien + PDF)
- [ ] F9 — Notation recette dans liste
- [ ] F10 — Conflits cross-module visible UI
- [ ] F11 — Vue calendrier mensuel planning
- [ ] F12 — Export planning PDF depuis UI
- [ ] F13 — Interface backtest stratégies paris
- [ ] F14 — Détection patterns jeux (hot hand)

Sprint 12 — Architecture

- [ ] A1 — Splitter `maison.py`
- [ ] A2 — Splitter `famille.py`
- [ ] A3 — Uniformiser noms factory (français)
- [ ] A4 — Génération automatique types TypeScript
- [ ] A5 — Audit tables orphelines ORM ↔ SQL

Sprint 13 — WhatsApp + Email

- [ ] W1 — TODO IA régénération planning WhatsApp (L131)
- [ ] W2 — 7 nouvelles commandes WhatsApp
- [ ] W3 — Compléter dispatcher email SMTP
- [ ] W4 — Préférences canaux notification utilisateur

Sprint 14 — IA + Cron

- [ ] IA1 — Résumé famille contextualisé IA
- [ ] IA2 — Prédiction liste courses
- [ ] IA3 — Détection anomalies dépenses
- [ ] IA4 — Chat IA contextuel cross-module enrichi
- [ ] IA5 — Suggestions activités Jules contextuelles
- [ ] IA6 — Génération automations via IA
- [ ] IA7 — Prédiction péremption personnalisée
- [ ] J1 — Job `sync_google_calendar`
- [ ] J2 — Job `rapport_nutrition_jules`
- [ ] J3 — Job `alerte_stock_bas`
- [ ] J4 — Job `archive_batches_expires`
- [ ] J5 — Job `rapport_maison_mensuel`
- [ ] J6 — Job `sync_openFoodFacts`

Sprint 15 — Inter-modules

- [ ] IM1 — Cuisine × Jules : adaptation auto repas semaine
- [ ] IM2 — Jardin × Cuisine : stock récoltes → recettes
- [ ] IM3 — Jeux × Budget : alerte paris seuil semaine
- [ ] IM4 — Planning × Voyages : pause automatique
- [ ] IM5 — Energie × Cuisine : heures creuses → appareils
- [ ] IM6 — Garmin × Nutrition : ajustement macros
- [ ] IM7 — Garmin × Famille : dashboard santé
- [ ] IM8 — Jeux × Automations : gain → journal/budget
- [ ] IM9 — Diagnostics IA × Artisans : workflow réparation

Sprint 16 — Admin complet

- [ ] ADM1 — Endpoints admin manquants (12 endpoints)
- [ ] ADM2 — Pages frontend admin (3 à créer, 1 à compléter)

- [ ] ADM3 — Sécurité audit logs généralisée

Plan généré le 28 mars 2026 depuis `ANALYSE_COMPLETE.md` (sections 2→13 + annexes) — GitHub Copilot